

საქართველოს აგრარული უნივერსიტეტი



ს ა კ უ რ ს ო პ რ ო ე ქ ტ ი

4-2

სიცრუის დეტექტორი

შეასრულა:

საინჟინრო-ტექნოლოგიური ფაკულტეტის

მეოთხე კურსის სტუდენტმა ნათია კანდელაკმა

2026 წ.

სარჩევი

1. ანოტაცია.....	3
2. შესავალი.....	4
3. ელექტრული სქემა და მისი მუშაობის აღწერა	5
4. კოდის აღწერა	9
5. გამოყენებული დეტალების ჩამონათვალი.....	23

ანოტაცია

ჯანდაცვის მსოფლიო ორგანიზაციის მიხედვით, სტრესი არის ფსიქიკური დამაბულობის მდგომარეობა, რომელიც რთული სიტუაციების დროს წარმოიქმნება და შესაძლოა გამოწვიოს მრავალი ფიზიკური და ფსიქოლოგიური დარღვევა.

სიცრუის დეტექტირება ეფუძნება ადამიანის ფიზიოლოგიური რეაქციების გაზომვას, რომლებიც უნებლიეთ წარმოიქმნება ფსიქოლოგიური დამაბულობის - კერძოდ, მოტყუების მცდელობის დროს. ეს ცვლილებები, როგორებიცაა გულისცემის სიხშირის მომატება, კანის ელექტრული გამტარობის ცვლილება და ტემპერატურული რყევები, ასახავს ავტონომიური ნერვული სისტემის რეაქციას და წარმოადგენს პოლიგრაფიული გამოკვლევის ფიზიოლოგიურ საფუძველს.

პროექტის მიზანია შეიქმნას არაინვაზიური, ელექტრონულ სენსორებზე დაფუძნებული სიცრუის დეტექტირების სისტემა, რომელიც რეალურ დროში ახდენს ადამიანის ფიზიოლოგიური სიგნალების აღრიცხვას და მათ საფუძველზე აფასებს ფსიქოფიზიოლოგიური დამაბულობის დონეს. სისტემა იყენებს სამ ძირითად სენსორს: პულსის სენსორს - გულისცემის სიხშირისა და მისი ცვალებადობის მონიტორინგისთვის, კანის გალვანური შეღწევადობის სენსორს - ემოციური აგზნებულობის ერთ-ერთ ყველაზე მგრძნობიარე ინდიკატორს, და ტემპერატურის სენსორს - კანის ტემპერატურის მიკრო-ცვლილებების დასაფიქსირებლად.

სისტემა უზრუნველყოფს სიგნალების რეალურ დროში ვიზუალიზაციას მატლაბში, მნიშვნელოვანია აღინიშნოს, რომ სისტემა არ წარმოადგენს სამართლებრივი მტკიცებულების საშუალებას — ის ზომავს ფიზიოლოგიურ სტრესსა, რომელიც შეიძლება სხვადასხვა მიზეზით იყოს გამოწვეული.

შესავალი

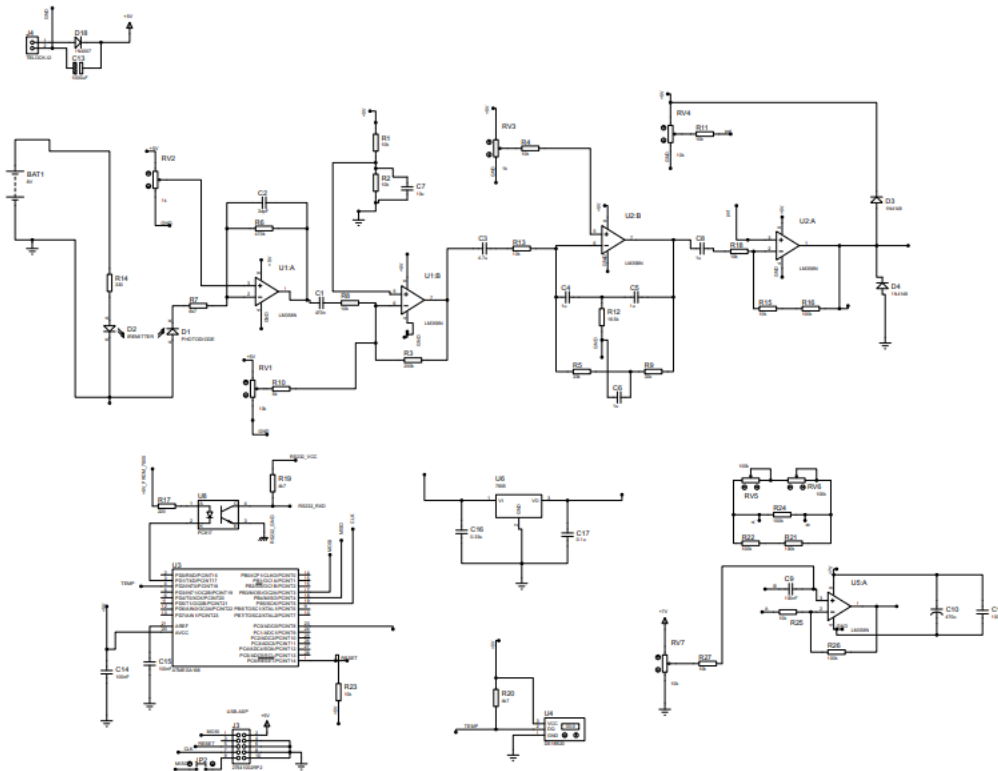
სიცრუე ადამიანური კომუნიკაციის განუყოფელი ნაწილია და მისი გამოვლენა საუკუნეების განმავლობაში წარმოადგენდა სამართალდამცავი, სასამართლო და ფსიქოლოგიური კვლევის ერთ-ერთ მთავარ გამოწვევას. ტრადიციული მეთოდები, როგორებიცაა ვიზუალური დაკვირვება და გამოკითხვის ტექნიკები, დამოკიდებულია ადამიანის სუბიექტურ შეფასებაზე და ხშირად არასანდოა. კვლევებმა აჩვენა, რომ ადამიანი საშუალოდ სიცრუეს მხოლოდ 54%-ის სიზუსტით ავლენს — რაც პრაქტიკულად შემთხვევითობის ტოლია.

მეოცე საუკუნის დასაწყისიდან მეცნიერებმა დაიწყეს ფიზიოლოგიური სიგნალების გამოყენება სიცრუის ობიექტური მაჩვენებლების სახით. ავტონომიური ნერვული სისტემა, რომელიც პასუხისმგებელია გულისცემაზე, კანის გამტარობაზე და ტემპერატურის რეგულაციაზე, მოტყუების მცდელობის დროს ქმნის გაზომვად, განმეორებად ფიზიოლოგიურ ცვლილებებს — ეს გახდა პოლიგრაფის, ანუ „სიცრუის დეტექტორის“, მეცნიერული საფუძველი.

ციფრული ტექნოლოგიებისა და ელექტრონული სენსორების განვითარებასთან ერთად, შესაძლებელი გახდა კომპაქტური, ხელმისაწვდომი და არაინვაზიური სისტემების შექმნა, რომლებიც რეალურ დროში ახდენენ მრავალი ფიზიოლოგიური პარამეტრის ერთდროულ გაზომვას და დამუშავებას. თანამედროვე მიკროკონტროლერები და სიგნალის დამუშავების ალგორითმები საშუალებას იძლევა ეს პროცესი განხორციელდეს პორტატულ მოწყობილობაში — საკვლევ ლაბორატორიასთან ან სპეციალიზებულ აღჭურვილობასთან დაკავშირების გარეშე.

წინამდებარე პროექტი მიზნად ისახავს სწორედ ასეთი სისტემის შექმნას: GSR, პულსისა და ტემპერატურის სენსორებზე დაფუძნებული მოწყობილობა, რომელიც ფიზიოლოგიური სიგნალების კომბინირებული ანალიზით განსაზღვრავს ფსიქოფიზიოლოგიური დაძაბულობის დონეს და ახდენს მის ვიზუალიზაციას რეალურ დროში. აღსანიშნავია, რომ სისტემა სამეცნიერო-საგანმანათლებლო მიზნებს ემსახურება და მისი შედეგები არ გამოდგება სამართლებრივ კონტექსტში მტკიცებულებად.

ელექტრული სქემა და მისი მუშაობის აღწერა



ელექტრული სქემის აღწერა:

წრედი მუშაობს 7 ვოლტზე, რადგანაც Atmega168-სთვის უსაფრთხო დასაშვები ძაბვა არის 5V, ვიყენებ 7805 ძაბვის რეგულატორს. პულსის გაზომვის სქემა დაფუძნებულია ფოტოპლეთიზმოგრაფიის (PPG) პრინციპზე. გამომსხივებელი შუქდიოდი განუწყვეტლივ ასხივებს ინფრაწითელ სინათლეს კანზე, ფოტოდიოდი კი იჭერს უკან არეკლილ სინათლეს. როდესაც გული იკუმშება, სისხლძარღვში სისხლის მოცულობა იზრდება, სისხლი ინფრაწითელ სინათლეს უფრო მეტად შთანთქმავს, შედეგად ფოტოდიოდადმიდ მიღებული სინათლის ინტენსივობა მცირდება. გულის მოდუნების ფაზაში პირიქით ხდება, სინათლის ინტენსივობა იზრდება. ეს პერიოდული ცვლილება ქმნის ანალოგიურ ელექტრულ სიგნალს, რომელიც პირდაპირ შეესაბამება გულისცემის რიტმს. ინფრაწითელ შუქდიოდთან მიმდევრობით მიერთებულია შეზღუდვის რეზისტორი, რათა დენი სტაბილური და უსაფრთხო იყოს. ფოტოდიოდი კი მიერთებულია საპირისპირო მიმართულებით, რაც მის მგრძნობელობას და რეაგირების სიჩქარეს მნიშვნელოვნად ზრდის.

ფოტოდiodიდან გამოსული სიგნალი ძალიან სუსტია, რამდენიმე მიკროვოლტიდან მილივოლტამდე და პირდაპირ დამუშავება შეუძლებელია. ამიტომ სიგნალი მიეწოდება U5:A - LM358N ოპერაციულ გამაძლიერებელს. RV5 და RV6 პოტენციომეტრები კალიბრაციის საშუალებას იძლევა სხვადასხვა კანის ტიპისა და პიგმენტაციისთვის, RV7 კი არეგულირებს სიგნალს, იმისათვის რომ ის მიკროკონტროლერის ADC-ს სამუშაო დიაპაზონში მოექცეს. R24 და R25 განსაზღვრავენ გამაძლიერებლის გაძლიერებას, R26 (1MΩ) კი უკუკავშირის რეზისტორია, რომელიც ასევე მაღალი სიხშირის კომპონენტების ფილტრაციას უწყობს ხელს.

პულსის გაზომვის სქემაში გამოყენებულია Twin-T Notch ფილტრი, რომელიც ოპერაციული გამაძლიერებლის უკუკავშირში ჩართვით Band-Pass ფილტრის ფუნქციას ასრულებს და 0.5–4 Hz სიხშირის დიაპაზონს ატარებს, რაც ზუსტად გულისცემის ფიზიოლოგიურ სიხშირეს შეესაბამება (30–240 დარტყმა/წუთი).

Twin-T ფილტრი შედგება ორი T-სებური განშტოებისგან. ზედა T-ს ქმნიან C4 და C5 (თითო 1μF) კონდენსატორები და R5 და R9 (თითო 33kΩ) რეზისტორები, ქვედა T-ს კი — C6 (1μF) კონდენსატორი და R12 (16.5kΩ) რეზისტორი. ფილტრის სიხშირე განისაზღვრება შემდეგი ფორმულით:

$$f = \frac{1}{2 * \pi * R * C} = \frac{1}{2 * \pi * 33000 * 0.000001} = 4.8 \text{ Hz}$$

მეორე საფეხურზე C8 (1μF) კონდენსატორი და R18 (10kΩ) რეზისტორი სიგნალს U2:A (LM358N)-ს მეორე გამაძლიერებელ საფეხურზე გადასცემენ. R15 (10kΩ) და R16 (100kΩ) რეზისტორები განსაზღვრავენ გამაძლიერებლის კოეფიციენტს.

გამოსასვლელზე D3 და D4 (1N4148) დიოდები დაცვის ფუნქციას ასრულებენ, ისინი ზღუდავენ გამოსასვლელ ძაბვას და უზრუნველყოფენ, რომ მიკროკონტროლერის ADC შესასვლელზე ძაბვამ არასდროს გადააჭარბოს 5 ვოლტს, რითაც იცავენ მიკროკონტროლერს დაზიანებისგან.

გაძლიერებული და გაფილტრული სიგნალი შემდეგ მიეწოდება მიკროკონტროლერის ADC შესასვლელს, სადაც ციფრულ მნიშვნელობად გარდაიქმნება და UART-ის მეშვეობით გადაეცემა MATLAB-ს დამუშავებისთვის. MATLAB-ში სიგნალზე გამოიყენება დამატებითი ციფრული ფილტრი და დეტექტირების ალგორითმი გულისცემის სიხშირის გამოსათვლელად.

რაც შეეხება გალვანური წინაღობის გაზომვას, სქემა დაფუძნებულია უიტსტონის ბოგის პრინციპზე. ბოგა შედგება RV5 და RV6 (თითო 100kΩ) პოტენციომეტრებისგან და R21, R22 (თითო 100kΩ) რეზისტორებისგან. R24 (100kΩ) რეზისტორი ხიდის ორ შტოს შორის განლაგებულია, A და B წერტილებს შორის კი მიერთებულია ელექტროდები, რომელთა შორის ადამიანის კანი წარმოადგენს ცვლად წინაღობას. RV5 და RV6 პოტენციომეტრები საშუალებას იძლევიან ბოგა დაბალანსდეს საწყის მდგომარეობაში. RV7 პოტენციომეტრი და R27 (10kΩ) რეზისტორი +7V კვების ძაბვიდან შუალედურ წერტილს ქმნიან, რათა ოპერაციული გამაძლიერებლის შემავალი სიგნალი სწორ სამუშაო დიაპაზონში მოექცეს.

ბოგას A წერტილიდან სიგნალი მიეწოდება U5:A (LM358N) ოპერაციული გამაძლიერებლის ინვერსიულ შესასვლელზე R25 (10kΩ) რეზისტორის მეშვეობით, B წერტილიდან კი, არაინვერსიულ შესასვლელზე C9 (100nF) კონდენსატორის მეშვეობით. C9 კონდენსატორი დამატებით ფილტრავს მაღალსიხშირულ ხმაურს. R26 (100kΩ) უკუკავშირის რეზისტორი განსაზღვრავს გამაძლიერებლის კოეფიციენტს:

$$G = \frac{R26}{R25} = \frac{100}{10} = 10$$

გამოსასვლელზე C10 (470μF) და C11 (100nF) კონდენსატორები პარალელურად გამოიყენება სიგნალის სტაბილიზაციისთვის - C10 დაბალსიხშირულ რყევებს ათანაბრებს, C11 კი მაღალსიხშირულ ხმაურს ფილტრავს. გაძლიერებული სიგნალი მიეწოდება მიკროკონტროლერის ADC შესასვლელს, სადაც ციფრულ მნიშვნელობად გარდაიქმნება და UART-ის მეშვეობით გადაეცემა MATLAB-ს დამუშავებისთვის.

კანის ელექტრული გამტარობა პირდაპირ კავშირშია ოფლის ჯირკვლების აქტივობასთან. სტრესის, შიშის ან ემოციური დაძაბულობის დროს ნერვული სისტემა ააქტიურებს ოფლის ჯირკვლებს, რაც იწვევს კანის ზედაპირზე ოფლის გამოყოფას. ოფლი ელექტროლიტებს შეიცავს და კანის ელექტრული გამტარობა იზრდება, შესაბამისად წინაღობა მცირდება, მშვიდ მდგომარეობაში კანის წინაღობა 10kΩ-დან რამდენიმე MΩ-მდე აღწევს, სტრესის პიკზე კი შეიძლება 1kΩ-მდეც დაეცეს. სწორედ ეს ცვლილება არღვევს უიტსტონის ბოგას ბალანსს და ქმნის ოპერაციული გამაძლიერებლის შემავალ დიფერენციალურ სიგნალს, რომელიც GSR-ის საზომ სიგნალს წარმოადგენს.

ტემპერატურის გაზომვის სქემა დაფუძნებულია DS18B20 ციფრულ სენსორზე, რომელიც 1-Wire პროტოკოლის საშუალებით უკავშირდება მიკროკონტროლერს. სენსორს სამი გამოსასვლელი აქვს: VCC (+5V კვება), GND და DQ (მონაცემთა ხაზი).

R20 (4.7k Ω) რეზისტორი გამოიყენება როგორც pull-up რეზისტორი, ის DQ ხაზს +5V-თან აკავშირებს, რაც 1-Wire პროტოკოლის სწორი მუშაობისთვის აუცილებელია.

DS18B20 ზომავს ტემპერატურას -55°C-დან +125°C-მდე დიაპაზონში, 0.5°C სიზუსტით. სხეულის ტემპერატურის გაზომვისთვის ეს სიზუსტე საკმარისია, რადგან სტრესის დროს კანის ტემპერატურა რამდენიმე გრადუსით იცვლება. ნერვული სისტემის გააქტიურებისას სისხლძარღვები ვიწროვდება, სისხლის მიმოქცევა კანის ზედაპირთან მცირდება და კანის ტემპერატურა ეცემა, ეს ცვლილება DS18B20-ის მიერ გაიზომება და 1-Wire პროტოკოლით გადაეცემა მიკროკონტროლერს, საიდანაც UART-ის მეშვეობით MATLAB-ს გადაეცემა დამუშავებისთვის.

კოდის აღწერა

```
#define F_CPU 8000000UL // პროცესორის სიხშირე – 8 მჰც
#include <avr/io.h> // I/O რეგისტრების განსაზღვრებები
#include <avr/interrupt.h> // წვეტის ბიბლიოთეკა
#include <util/delay.h>

#define BUF_SIZE 100u // ბუფერის ზომა – 100(2 წამი 50ჰც-ზე)

static volatile uint16_t ppg_buf1[BUF_SIZE]; // PPG-ის პირველი ბუფერი
static volatile uint16_t ppg_buf2[BUF_SIZE]; // PPG-ის მეორე ბუფერი
static volatile uint16_t gsr_buf1[BUF_SIZE]; // GSR-ის პირველი ბუფერი
static volatile uint16_t gsr_buf2[BUF_SIZE]; // GSR-ის მეორე ბუფერი

/* — ADC — */
static volatile uint8_t adc_active = 0; // რომელი ბუფერი ივსება (0 ან 1)
static volatile uint16_t adc_idx = 0; // ბუფერში მიმდინარე ნიმუშის ინდექსი
static volatile uint8_t adc_channel = 0; // ADC არხი: 0 = PPG, 1 = GSR

/* — TX — */
static volatile uint8_t tx_busy = 0; // გადაცემა მიმდინარეობს თუ არა (1 = დაკავებული)
static volatile uint8_t tx_active = 0; // რომელი ბუფერი იგზავნება ამჟამად
static volatile uint16_t tx_idx = 0; // გადასაცემი ნიმუშის ინდექსი
/*
 * tx_phase: // გადაცემის ფაზა (7 ბაიტის პაკეტი):
 * 0 = send 0xAA header
 * 1 = PPG high byte // 1 – PPG მნიშვნელობის მაღალი ბაიტი
 * 2 = PPG low byte // 2 – PPG მნიშვნელობის დაბალი ბაიტი
 * 3 = GSR high byte // 3 – GSR მნიშვნელობის მაღალი ბაიტი
 * 4 = GSR low byte // 4 – GSR მნიშვნელობის დაბალი ბაიტი
 * 5 = TEMP high byte // 5 – ტემპერატურის მაღალი ბაიტი
 * 6 = TEMP low byte → then advance tx_idx // 6 – ტემპერატურის დაბალი ბაიტი,
შემდეგ tx_idx იზრდება
 */
static volatile uint8_t tx_phase = 0; // გადაცემის მიმდინარე ფაზა (0-5)

/* — ტემპერატურა — */
static volatile int16_t last_temp_raw = 0; // DS18B20-ის ბოლო წაკითხული 16-ბიტის მნიშვნელობა

//DS18B20 – 1-Wire (bit-bang on PD2) // DS18B20 ტემპერატურის სენსორი – 1-
Wire პროტოკოლი PD2 პინზე
#define OW_DDR DDRD // 1-Wire მიმართულების რეგისტრი
(DDRD)
#define OW_PORT PORTD // 1-Wire გამოსავლის პორტი (PORTD)
```

```

#define OW_PIN    PIND                                // 1-Wire შეყვანის ფეხის წაკითხვა
(PIND)
#define OW_BIT    PD2                                // 1-Wire სიგნალის ფეხი – PD2

static inline void ow_low(void) { OW_DDR |= (1 << OW_BIT); OW_PORT &= ~(1 << OW_BIT);
} // დაბალ დონეზე (0) დაყენება
static inline void ow_release(void) { OW_DDR &= ~(1 << OW_BIT); OW_PORT |= (1 <<
OW_BIT); } // ფეხის გათავისუფლება (შეყვანა + pull-up)
static inline uint8_t ow_read_bit(void) { return (OW_PIN >> OW_BIT) & 1; }
// ფეხის მდგომარეობის წაკითხვა (0 ან 1)

// აბრუნებს 1-ს თუ მოწყობილობა დაკავშირებულია
static uint8_t ow_reset(void)
{
    ow_low();                                //Reset პულსი იწყება
    _delay_us(480);                          // 480 მიკროწამი ლოდინი (1-Wire
სპეციფიკაცია)
    ow_release();                            // გაშვება – სენსორი პასუხობს
    _delay_us(70);                          // 70 მიკროწამი – Presence pulse-ის
ლოდინი
    uint8_t presence = !ow_read_bit();       // დაბლაა? – სენსორი დასწრებას
ადასტურებს
    _delay_us(410);                          // 410 მიკროწამი – Reset ციკლის
დასრულება
    return presence;                         // 1 = სენსორი აღმოჩენილია, 0 = არ
არის
}

static void ow_write_bit(uint8_t bit)        // ერთი ბიტის გაგზავნა 1-Wire
პროტოკოლით
{
    ow_low();                                // დაწყება
    if (bit) { _delay_us(6); ow_release(); _delay_us(64); } // "1" ბიტი: 6მიკროწმ
დაბლა, შემდეგ გათავისუფლება
    else { _delay_us(60); ow_release(); _delay_us(10); } // "0" ბიტი: 60მიკროწმ
დაბლა, შემდეგ გათავისუფლება
}

static uint8_t ow_read_bit_val(void)        // ერთი ბიტის წაკითხვა 1-Wire-დან
{
    ow_low();                                // წაკითხვის სლოტის დაწყება
    _delay_us(6);                          // 6 მიკროწამი – სენსორი ემზადება
    ow_release();                          // გათავისუფლება – სენსორი სიგნალს
გასცემს
    _delay_us(9);                          // 9 მიკროწამი – სიგნალის
სტაბილიზაცია
    uint8_t val = ow_read_bit();           // პინის მნიშვნელობის წაკითხვა
    _delay_us(55);                          // 55 მიკროწამი – სლოტის დასრულება
    return val;                            // წაკითხული ბიტის დაბრუნება
}

static void ow_write_byte(uint8_t byte)    // ერთი ბაიტის (8 ბიტის) გაგზავნა 1-
Wire-ით

```

```

{
    for (uint8_t i = 0; i < 8; i++) { ow_write_bit(byte & 1); byte >>= 1; } // LSB-დან
დაწყებით თითოეული ბიტი იგზავნება
}

static uint8_t ow_read_byte(void) // ერთი ბაიტის (8 ბიტის) წაკითხვა 1-
Wire-დან
{
    uint8_t val = 0; // შედეგი ნულიდან იწყება
    for (uint8_t i = 0; i < 8; i++) { if (ow_read_bit_val()) val |= (1 << i); } //
თითოეული ბიტი LSB-დან ჩაიწერება
    return val;
}

// კონვერსიის დაწყება (არ ბლოკავს – async)
// ow_read_temp() მინიმუმ 750ms-ის შემდეგ გამოიძახეთ

static void ow_start_conversion(void)
{
    ow_reset(); // 1-Wire Reset – სენსორის
ინიციალიზაცია

    ow_write_byte(0xCC); // SKIP ROM ბრძანება – ყველა
მოწყობილობას ეხება

    ow_write_byte(0x44); // CONVERT T ბრძანება – ტემპერატურის
გაზომვა იწყება
}

// Scratchpad მესხიერების წაკითხვა, 16-ბიტი მნიშვნელობა
static int16_t ow_read_temp(void)
{
    ow_reset(); // 1-Wire Reset
    ow_write_byte(0xCC); // SKIP ROM – ყველა სენსორს ეხება
    ow_write_byte(0xBE); // READ scratchpad-ის წაკითხვა
    uint8_t lo = ow_read_byte(); // ტემპერატურის დაბალი ბაიტი
    uint8_t hi = ow_read_byte(); // ტემპერატურის მაღალი ბაიტი
    return (int16_t)((uint16_t)hi << 8 | lo); // ორი ბაიტი გაერთიანება 16-ბიტ
მნიშვნელობად
}

// დამხმარე ფუნქციები – ბუფერის მაჩვენებლები

static inline volatile uint16_t* ppg_buf(uint8_t which)
{ return (which == 0) ? ppg_buf1 : ppg_buf2; } // 0 = ppg_buf1, 1 = ppg_buf2 – სწორი
PPG ბუფერის დაბრუნება

static inline volatile uint16_t* gsr_buf(uint8_t which)
{ return (which == 0) ? gsr_buf1 : gsr_buf2; } // 0 = gsr_buf1, 1 = gsr_buf2 – სწორი
GSR ბუფერის დაბრუნება

// TIMER0 – 50 ჰც-ზე ADC-ს ააქტიურებს
ISR(TIMER0_COMPA_vect)
{
    // პირველად PPG (ADC0)

```

```

    ADMUX = (1 << REFS0) | 0x00; // AVcc ეტალონი, არხი 0 (PPG)
    ADCSRA |= (1 << ADSC); // ADC კონვერსიის დაწყება
    adc_channel = 0; // PPG-ს ვზომავ
}

ISR(ADC_vect) // ADC კონვერსიის დასრულების შეწყვეტა
{
    uint16_t result = ADC; // ADC-ის შედეგის წაკითხვა

    if (adc_channel == 0) // თუ ახლა PPG-ს ვზომავდით
    {
        // PPG ჩაიწერება, მაშინვე GSR-ი იზომება

        ppg_buf(adc_active)[adc_idx] = result; // PPG მნიშვნელობა ბუფერში
        ADMUX = (1 << REFS0) | 0x01; // AVcc ეტალონი, არხი 1 (GSR)
        ADCSRA |= (1 << ADSC); // GSR კონვერსიის დაწყება
        adc_channel = 1; // მდგომარეობა: ახლა GSR-ს ვზომავთ
    }
    else
    {
        // GSR ჩაიწერება, ინდექსი იზრდება
        gsr_buf(adc_active)[adc_idx] = result; // GSR მნიშვნელობა ბუფერში
        adc_idx++; // შემდეგ ნიმუშზე გადასვლა

        if (adc_idx >= BUF_SIZE) // თუ ბუფერი გაივსო (100 ნიმუში)
        {
            adc_idx = 0; // ინდექსის გადატვირთვა
            if (!tx_busy) // თუ UART გადაცემა არ მიმდინარეობს
            {
                tx_active = adc_active; // გავსებული ბუფერი გადასაცემად

                // მეორე (ცარიელ) ბუფერზე გადართვა
                // გადაცემის ინდექსის გადატვირთვა
                // გადაცემის ფაზა – თავიდან
                // UART დაკავებულია
                // პირველი ბაიტი (0xAA) TX ISR-ს
                // მიმდინარე PPG ბუფერის მაჩვენებელი
                // მიმდინარე GSR ბუფერის მაჩვენებელი

                adc_active ^= 1;
                tx_idx = 0;
                tx_phase = 0;
                tx_busy = 1;
                UDR0 = 0xAA;
            }
        }
    }
}

// USART TX – თითო ნიმუშზე 7-ბაიტიათი პაკეტი იგზავნება
// [0xAA] [PPG_H] [PPG_L] [GSR_H] [GSR_L] [TEMP_H] [TEMP_L]
ISR(USART_TX_vect) // UART გადაცემის დასრულების შეწყვეტა
{
    if (!tx_busy) return; // თუ გასაგზავნი არ არის – გამოსვლა

    volatile uint16_t *pb = ppg_buf(tx_active);
    volatile uint16_t *gb = gsr_buf(tx_active);

```

```

switch (tx_phase)                                     // გადაცემის ფაზის მიხედვით
მოქმედება
{
    case 0:                                           // ფაზა 0: PPG-ის მაღალი ბაიტი
        UDR0 = (uint8_t)(pb[tx_idx] >> 8);          // PPG[15:8] – მაღალი ბაიტი UART-ზე
        tx_phase = 1;                                // შემდეგი ფაზა
        break;

    case 1: // ფაზა 1: PPG-ის დაბალი ბაიტი
        UDR0 = (uint8_t)(pb[tx_idx] & 0xFF);         // PPG[7:0] – დაბალი ბაიტი UART-ზე
        tx_phase = 2;                                // შემდეგი ფაზა
        break;

    case 2:                                           // ფაზა 2: GSR-ის მაღალი ბაიტი
        UDR0 = (uint8_t)(gb[tx_idx] >> 8);          // GSR[15:8] – მაღალი ბაიტი UART-ზე
        tx_phase = 3;                                // შემდეგი ფაზა
        break;

    case 3:                                           // ფაზა 3: GSR-ის დაბალი ბაიტი
        UDR0 = (uint8_t)(gb[tx_idx] & 0xFF);         // GSR[7:0] – დაბალი ბაიტი UART-ზე
        tx_phase = 4;                                // შემდეგი ფაზა
        break;

    case 4:                                           // ფაზა 4: ტემპერატურის მაღალი ბაიტი
        UDR0 = (uint8_t)((uint16_t)last_temp_raw >> 8); // TEMP[15:8] UART-ზე
        tx_phase = 5;                                // შემდეგი ფაზა
        break;

    case 5: // ფაზა 5: ტემპ. დაბალი ბაიტი, შემდეგ ნიმუშზე გადასვლა
        UDR0 = (uint8_t)((uint16_t)last_temp_raw & 0xFF); // TEMP[7:0] UART-ზე
        tx_phase = 0;                                // ფაზა – ნულზე დაბრუნება
        pb[tx_idx] = 0;                               // PPG ბუფერის გაწმენდა
        gb[tx_idx] = 0;                               // GSR ბუფერის გაწმენდა
        tx_idx++;                                     // შემდეგ ნიმუშზე გადასვლა
        if (tx_idx >= BUF_SIZE)                       // თუ ყველა ნიმუში გაიგზავნა
        {                                             // გადაცემა დასრულდა
            tx_busy = 0;
        }
        else
        {
            UDR0 = 0xAA;
        }
        break;
    }
}

// პერიფერიული მოწყობილობების ინიციალიზაცია
static void uart_init(void)                          // UART-ის ინიციალიზაციის ფუნქცია
{
    UBRR0H = 0x00;                                    // Baud rate რეგისტრის მაღალი
    UBRR0L = 0x33;                                    // Baud rate: 9600 bps @ 8MHz (0x33
= 51)
    UCSR0C = 0x06;                                    // 8 ბიტი, parite არ არის, 1 stop
ბიტი)

```

```

    UCSR0A = 0x00; // სტატუსის რეგისტრი
    UCSR0B = (1 << TXCIE0) | (1 << TXEN0); // TX ჩართვა + TX-complete შეწყვეტის
ჩართვა
}

static void adc_init(void) // ADC-ის ინიციალიზაციის ფუნქცია
{
    ADCSRA = (1 << ADEN) | (1 << ADIF) // ADC ჩართვა, შეწყვეტა ჩართვა,
    | (1 << ADPS2) | (1 << ADPS1); // ეტალონი: AVcc, არხი 0
    prescaler /64 // PC0 და PC1 ციფრული ბუფერების
    ADMUX = (1 << REFS0);
    DIDR0 = (1 << ADC0D) | (1 << ADC1D);
    გამორთვა (ენერგიის დაზოგვა)
}

static void timer_init(void) // Timer0-ის ინიციალიზაციის ფუნქცია
{
    TCCR0A = (1 << WGM01); // CTC რეჟიმი (OCR0A-ზე გადატვირთვა)
    TCCR0B = (1 << CS02); // Prescaler /256
    OCR0A = 155; // 8,000,000 / 256 / 156 ≈ 50 ჰც
    TCNT0 = 0; // Timer-ის მრიცხველის გადატვირთვა
    TIMSK0 = (1 << OCIE0A); // OCR0A შეწყვეტის ჩართვა (50 ჰც)
}

int main(void)
{
    /* PC0, PC1 – inputs (ADC) */ // PC0, PC1 – შეყვანა (ADC სენსორები)
    DDRC = 0x00;
    PORTC = 0x00; // pull-up გამორთვა C პორტზე

    // PD2 – 1-Wire (მიმართულება ow_ ფუნქციებს მართავს)

    ow_release(); // PD2 + pull-up

    timer_init(); // Timer0 ინიციალიზაცია (50 ჰც)
    uart_init(); // UART ინიციალიზაცია (9600 baud)
    adc_init(); // ADC ინიციალიზაცია

    // პირველი ტემპ. გაზომვა (შედეგი ~750ms-ში მოდის)
    ow_start_conversion(); // DS18B20

    sei(); // გლობალური წვეტების ჩართვა

    uint8_t prev_tx_busy = 0; // წინა ციკლის tx_busy მნიშვნელობა

    while (1)
    {
        // შემდეგი პაკეტი განახლებულ ტემპ. მიიღებს

        uint8_t cur = tx_busy; // მიმდინარე tx_busy მდგომარეობა
        if (prev_tx_busy && !cur) // თუ tx_busy 1→0 გადაიცვალა
        {
            //დასრულდა → ძველი კონვერსიის შედეგი

```

```

        last_temp_raw = ow_read_temp();           // DS18B20-ის ბოლო ტემპერატურის
წაკითხვა
        ow_start_conversion();                   // DS18B20 კვლავ იწყებს გაზომვას
    }
    prev_tx_busy = cur;                          // მდგომარეობის შენახვა მომდევნო
ციკლისთვის
}
}

```

სენსორებიდან მიღებულ ინფორმაციას ვაგზავნი MATLAB-ში, რადგან მიკროკონტროლერს არ გააჩნია საკმარისი გამოთვლითი სიმძლავრე სიგნალის დასამუშავებლად და ასევე მატლაბს აქვს ჩაშენებული ფუნქციები, რომელთა გამოყენებით შევძელი მარტივად ჩამეტარებინა მათემატიკური გამოთვლები, ამიტომ მონაცემები მატლაბში იგზავნება, სადაც იგი ასრულებს ყველა რთულ გამოთვლას.

```

clear; clc;                                     % სამუშაო სივრცის და კონსოლის გაწმენდა

delete(instrfindall);                          % ყველა ღია serial პორტის დახურვა (თუ ადრე
გახსნილი იყო)

```

```

s = serial('COM4', 'BaudRate', 9600, 'Timeout', 10);

```

```

% COM4 serial პორტის შექმნა: 9600 baud, 10 წმ timeout

```

```

fopen(s);                                       % serial პორტის გახსნა (კავშირის დაწყება)

```

```

Fs = 50;                                       % შერჩევის სიხშირე - 50 ჰც (ყოველ 20ms ერთი
ნიმუში)

```

```

BUF_SIZE = 100;                              % ბუფერის ზომა — 100 (2 წამის ფანჯარა)

```

```

t = (0 : BUF_SIZE-1) / Fs;                   % დროის ვექტორი წამებში: 0, 0.02, 0.04, ..., 1.98

```

```

ppg_data = zeros(1, BUF_SIZE);               % PPG-ის მასივი — ნულებით
ინიციალიზაცია

```

```

gsr_data = zeros(1, BUF_SIZE);           % GSR-ის მასივი — ნულებით
ინიციალიზაცია

temp_data = zeros(1, BUF_SIZE);          % ტემპ. მასივი — ერთი მნიშვნელობა
პაკეტი, °C

f_low = 0.5;                             % ზოლოვანი ფილტრის ქვედა სიხშირე: 0.5 ჰც
(გულისცემა მიწ.)

f_high = 4.0;                             % ზოლოვანი ფილტრის ზედა სიხშირე: 4.0 ჰც
(გულისცემა მაქს.)

[b, a] = butter(4, [f_low f_high] / (Fs/2), 'bandpass');
% მე-4 რიგის Butterworth ფილტრი

fig = figure('Name', 'Lie Detector', 'NumberTitle', 'off'); % გრაფიკის ფანჯრის შექმნა

% --- PPG-ის ნედლი სიგნალი ---
ax1 = subplot(4,1,1);                     % პირველი სუბგრაფიკი (4-დან 1)
h_ppg = plot(ax1, t, ppg_data, 'b');      % PPG-ის სიგნალი ლურჯად
title(ax1, 'PPG — Raw');                 % სათაური
xlabel(ax1, 'Time (s)'); ylabel(ax1, 'Voltage (V)'); % ღერძების წარწერები
ylim(ax1, [0 5]);

% Y ღერძი: 0–5 ვოლტი (ADC-ის სრული დიაპაზონი)

ax2 = subplot(4,1,2);                     % მეორე სუბგრაფიკი
h_pfilt = plot(ax2, t, ppg_data, 'g'); hold(ax2,'on'); % გაფილტრული PPG მწვანედ
h_peaks = plot(ax2, NaN, NaN, 'rv', 'MarkerFaceColor','r');
title(ax2, 'PPG — Filtered');

```

```

xlabel(ax2, 'Time (s)'); ylabel(ax2, 'Voltage (V)');
legend(ax2, 'PPG filtered','Peaks');

% --- GSR ---
ax3 = subplot(4,1,3); % მესამე სუბგრაფიკი
h_gsr = plot(ax3, t, gsr_data, 'm');
title(ax3, 'GSR — Skin Conductance');
xlabel(ax3, 'Time (s)'); ylabel(ax3, 'Voltage (V)');
ylim(ax3, [0 5]); % Y დერძი: 0–5 ვოლტი

% --- ტემპერატურის სიგნალი ---
ax4 = subplot(4,1,4);
h_temp = plot(ax4, t, temp_data, 'r');
title(ax4, 'Skin Temperature (°C)');
xlabel(ax4, 'Time (s)'); ylabel(ax4, '°C');
ylim(ax4, [25 40]);

for i = 1 : BUF_SIZE % ყოველი ნიმუშის ციკლი (100-ჯერ)

    sync = 0;
    while sync ~= hex2dec('AA') % სანამ 0xAA ბაიტი არ მოვა
        raw = fread(s, 1, 'uint8'); % ერთი ბაიტის წაკითხვა serial-დან
        if isempty(raw), continue; end % თუ არ მოვიდა — გაგრძელება
        sync = raw; % წაკითხული ბაიტი სინქროში ჩაწერა
    end

```

```

% --- 6 მონაცემთა ბაიტის წაკითხვა ---

raw6 = fread(s, 6, 'uint8');          % 6 ბაიტი ერთბაშად (PPG_H, PPG_L, GSR_H,
GSR_L, TEMP_H, TEMP_L)

if numel(raw6) < 6, continue; end      % თუ 6-ზე ნაკლები მოვიდა —
გამოვტოვოთ

ppg_raw = bitor(bitshift(uint16(raw6(1)), 8), uint16(raw6(2)));
% PPG: მაღალი<<8 | დაბალი → 16-ბიტი

gsr_raw = bitor(bitshift(uint16(raw6(3)), 8), uint16(raw6(4)));
% GSR: მაღალი<<8 | დაბალი → 16-ბიტი

temp_raw = typecast(uint16(bitor(bitshift(uint16(raw6(5)),8), ...
uint16(raw6(6)))), 'int16');          % TEMP: uint16 → int16 (ნიშნის)

% ფიზიკურ ერთეულებში გადაყვანა
ppg_data(i) = double(ppg_raw) * 5.0 / 1023.0; % PPG: ADC (0-1023) → ვოლტი (0-5V)
gsr_data(i) = double(gsr_raw) * 5.0 / 1023.0; % GSR: ADC (0-1023) → ვოლტი (0-5V)
temp_data(i) = double(temp_raw) * 0.0625;      % ტემპ.: DS18B20 ნედლი × 0.0625 = °C
(1 LSB = 0.0625°C)

% --- PPG-ის გრაფიკის განახლება ---

set(h_ppg, 'YData', ppg_data);

% --- PPG-ის ფილტრაცია და მწვერვალების პოვნა ---

if i >= 30                                % მინიმუმ 30 ნიმუში საჭიროა ფილტრაციისთვის

```

```

ppg_filt = filtfilt(b, a, ppg_data(1:i)); % Butterworth ფილტრი (ნულოვანი
ფაზური გვიანება)

ppg_smooth = movmean(ppg_filt, 5); % 5-ნიმუშიანი მოძრავი საშუალო

partial = zeros(1, BUF_SIZE); % დროებითი სრული სიგრძის ვექტორი
partial(1:i) = ppg_smooth; % გაფილტრული ნაწილი ჩაიწერება
set(h_pfilt, 'YData', partial); % გაფილტრული PPG-ის გრაფიკი განახლება

min_dist = round(0.4 * Fs); % პიკებს შორის მინ. დისტანცია: 0.4წმ = 20
ნიმუში (max 150 BPM)

min_prom = 0.25 * (max(ppg_smooth) - min(ppg_smooth)); % მინ. სიმაღლე:
სიგნალის ამპლიტუდის 25%

try

[pv, pl] = findpeaks(ppg_smooth, ... % პიკების პოვნა
'MinPeakDistance', min_dist, ... % მინ. დისტანცია
'MinPeakProminence', min_prom); % მინ. სიმაღლე
set(h_peaks, 'XData', t(pl), 'YData', pv);

if length(pl) >= 2 % მინ. 2 პიკია საჭიროა BPM-ისთვის
rr = diff(pl) / Fs; % RR ინტერვალები წამებში პიკებს შორის დრო)
bpm = 60 ./ rr; % წუთში გულისცემები = 60 / RR
bpm = bpm(bpm >= 45 & bpm <= 150); % არარეალური მნიშვნელობების
გაფილტვრა (45–150 BPM)
if ~isempty(bpm)
title(ax2, sprintf('PPG Filtered — BPM: %.1f', median(bpm)));
end

```

```

        end
    catch
    end
end

% --- GSR-ის გრაფიკის განახლება ---
set(h_gsr, 'YData', gsr_data);

set(h_temp, 'YData', temp_data);
if temp_data(i) > 0 % თუ ვალიდური ტემპ. მოვიდა (არა ნული)
    title(ax4, sprintf('Skin Temperature — %.2f °C', temp_data(i)));
end

drawnow limitrate;

% გრაფიკების ეკრანზე გამოტანა (შეზღუდული სიხშირით — სიჩქარისთვის)
end

ppg_filt = filtfilt(b, a, ppg_data); % მთელი PPG სიგნალის ფილტრაცია
ppg_smooth = movmean(ppg_filt, 5); % მოძრავი საშუალო — გაგლუვება

min_dist = round(0.4 * Fs); % მინ. მანძილი პიკებს შორის (0.4 წმ)
min_prom = 0.25 * (max(ppg_smooth) - min(ppg_smooth));
% მინ. სიმაღლე (ამპლიტუდის 25%)

[peak_vals, peak_locs] = findpeaks(ppg_smooth, ... % საბოლოო პიკების პოვნა
'MinPeakDistance', min_dist, ...

```

```

'MinPeakProminence', min_prom);

% 60%-იანი ამპლიტუდის წესი — არარეალური მწვერვალების მოშორება
valid_locs = zeros(1, length(peak_locs));
valid_vals = zeros(1, length(peak_locs));    % ვალიდური პიკების მნიშვნელობები
count = 0;                                   % ვალიდური პიკების რაოდენობა
for k = 1 : length(peak_locs)                % ყველა ნაპოვნი პიკების შემოწმება
    if count == 0 || peak_vals(k) >= 0.6 * valid_vals(count)
% პირველი ან წინა 60%-ზე მეტი
        count = count + 1;                   % ვალიდური პიკების დამატება
        valid_locs(count) = peak_locs(k);    % პოზიციის შენახვა
        valid_vals(count) = peak_vals(k);    % მნიშვნელობის შენახვა
    end
end
peak_locs = valid_locs(1:count);              % მხოლოდ ვალიდური პოზიციები
peak_vals = valid_vals(1:count);              % მხოლოდ ვალიდური მნიშვნელობები

set(h_pfilt, 'YData', ppg_smooth);            % გაფილტრული PPG-ის საბოლოო გრაფიკი
set(h_peaks, 'XData', t(peak_locs), 'YData', peak_vals); % საბოლოო პიკების გრაფიკი

if length(peak_locs) >= 2                    % მინ. 2 პიკია საჭიროა გამოთვლებისთვის
    RR = diff(peak_locs) / Fs;               % RR ინტერვალები წამებში
    bpm = 60 ./ RR;                           % გულისცემები წუთში
    bpm = bpm(bpm >= 45 & bpm <= 150);       % არარეალური მნიშვნელობების
    მოშორება
    if ~isempty(bpm)

```

```

    fprintf('Detected %d heartbeats\n', count);           % გამოვლენილი
გულისცემების რაოდენობა

    fprintf('Average Heart Rate : %.1f BPM\n', median(bpm)); % საშუალო
გულისცემა (BPM)

    fprintf('Average GSR      : %.3f V\n', mean(gsr_data)); % GSR-ის საშუალო
მნიშვნელობა

    fprintf('Average Temperature: %.2f C\n', mean(temp_data(temp_data>0))); % კანის
საშუალო ტემპერატურა

    end

else

    disp('Not enough peaks detected. Check signal quality.');
```

```

end

fclose(s); delete(s); clear s;           % serial პორტის დახურვა, ობიექტის წაშლა,
ცვლადის გაწმენდა

```

გამოყენებული დეტალების ჩამონათვალი

1. ATmega168
2. RS232
3. LM358N (3)
4. DS18B20
5. ინფრაწითელი გამომსხივებელი
6. ფოტოდiodი
7. ელექტროდები
8. 7805
9. დიოდი 1N4148 (2)
10. ცვლადი წინაღობა 10K (4), 100k (2)
11. წინაღობა 1k, 16.5k, 10k, 33k, 100k, 4.7k
12. კონდენსატორი 4.7uF, 1uF, 100nF, 40uF
13. კონექტორი 10 ფეხიანი
14. სამონტაჟო დაფა
15. კვების წყარო